

LINUX

BUILD THE FOUNDATION.
POWER YOUR DEVOPS JOURNEY.

Essential Linux skills for DevOps Engineers



COMMAND LINE
ESSENTIALS



FILES &
DIRECTORIES



SECURITY &
PERMISSIONS



SERVERS &
AUTOMATION

```
$ ls -l /home/devops  
$ pwd  
/home/devops  
$ systemctl status nginx  
$ ps aux | grep java  
$ df -h  
$ git status  
$ docker ps -a  
$ uptime
```



www.tech365.ng



info@tech365.ng



+2348029704730



@tech365ng

1. INTRODUCTION TO LINUX

Linux is a powerful, open-source operating system that forms the foundation of modern DevOps. It runs servers, cloud platforms, containers, and most of the tools DevOps engineers use every day.



```
user@linux:~$ whoami
devops
user@linux:~$ uname -a
Linux server 6.5.0-101-generic
user@linux:~$
```



WHAT IS LINUX?

Linux is a free and open-source operating system built around the Linux kernel. It is stable, secure, flexible and highly customizable. It supports everything from small devices to enterprise servers and supercomputers.



KEY CONCEPTS



Kernel

Core of the operating system that manages hardware and processes.



Shell

Command-line interface that lets you interact with the system.



Terminal

Program that provides access to the command line.



CLI

Command Line Interface used to give instructions to the system.



Open Source

Linux is free to use, modify and distribute.



WHY DEVOPS ENGINEERS NEED LINUX

- ✓ Most servers and cloud instances run Linux.
- ✓ DevOps tools (Docker, Kubernetes, Jenkins, Terraform, Ansible) are built for Linux.
- ✓ Better automation, scripting and remote server management.
- ✓ Essential for working in cloud environments.
- ✓ High demand skill for DevOps and SRE roles.



WHERE IS LINUX USED?

- Web Servers
- Cloud Platforms (AWS, Azure, GCP)
- Containers & Kubernetes
- CI/CD Pipelines
- Databases
- DevOps Tools & Automation
- Supercomputers and Research



POPULAR LINUX DISTRIBUTIONS



Ubuntu

User-friendly, widely used in servers and desktops.



Debian

Stable and chosen as the base for many distributions.



Red Hat Enterprise Linux

Enterprise-grade, used in many organizations.



Rocky Linux

Enterprise-compatible alternative to RHEL.



Amazon Linux

Optimized for AWS environments.



Your first Linux command

Open the terminal and try:

```
ls
```

Lists files and directories in the current directory.



Pro Tip

Linux is case-sensitive. "File.txt" is different from "file.txt".

LINUX vs WINDOWS vs macOS

THREE POWERFUL OPERATING SYSTEMS. DIFFERENT BY DESIGN. BUILT FOR DIFFERENT NEEDS.



LINUX



OPEN SOURCE

Free and open-source. Anyone can use, modify, and distribute.



SECURITY

Highly secure with robust permission model and strong community support.



PERFORMANCE

Lightweight and efficient. Runs well on low-end and high-end systems.



CUSTOMIZATION

Highly customizable. Full control over the system.



SOFTWARE

Huge repository of open-source tools. Great for developers and DevOps.



BEST FOR

Servers, cloud, DevOps, development, automation, and power users.



WINDOWS



LICENSE

Proprietary software. Requires license to use.



SECURITY

Secure with regular updates and built-in protection.



PERFORMANCE

Optimized for a wide range of hardware and use cases.



CUSTOMIZATION

User-friendly with many customization options.



SOFTWARE

Wide range of commercial software and games.



BEST FOR

Business, office productivity, gaming, and general users.



macOS



DESIGNED BY APPLE

Proprietary and available only on Apple hardware.



SECURITY

Built with strong security and privacy in mind.



PERFORMANCE

Optimized for Apple hardware (Mac) for high performance.



USER EXPERIENCE

Sleek design, intuitive and easy to use.



SOFTWARE

Great tools for creatives, developers, and professionals.



BEST FOR

Design, video editing, development, and creative professionals.



THE BOTTOM LINE

Linux powers the world behind the scenes. Windows powers productivity and entertainment for millions. macOS powers creativity and innovation. Choose the right tool for the right job.



FOR DEVOPS ENGINEERS

Linux is the foundation. Learn it deeply. It powers servers, clouds, containers, and most DevOps tools.



www.tech365.ng



info@tech365.ng



+2348029704730



@tech365ng

LINUX KERNEL

The Linux Kernel is the core of the Linux operating system. It acts as a bridge between the hardware and the applications, managing system resources and providing essential services.

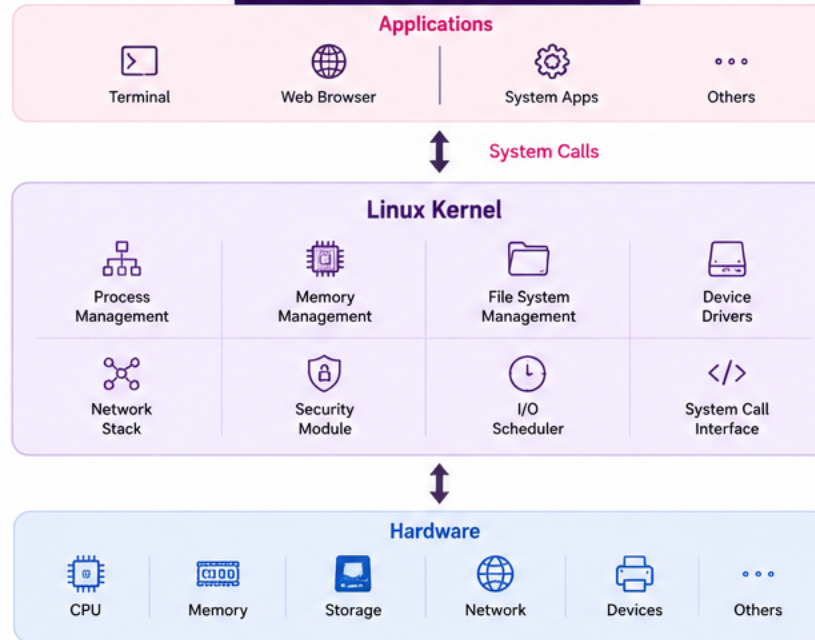


The kernel is the heart of Linux. It handles everything between your applications and the hardware.

– **Linus Torvalds**
Creator of Linux



HOW LINUX KERNEL WORKS



KEY RESPONSIBILITIES



Resource Management

Manages CPU, memory, devices and all system resources.



Process Management

Handles process creation, scheduling, synchronization and termination.



Memory Management

Manages physical and virtual memory, including allocation and protection.



Device Management

Controls device drivers and handles I/O operations.



Security & Protection

Ensures system security, access control and isolation.



System Calls Interface

Provides an interface for applications to request kernel services.



KEY FACTS

- ✓ Open source and community-driven.
- ✓ Runs in kernel space (privileged mode).
- ✓ First released in 1991 by Linus Torvalds.
- ✓ Supports multiple architectures.
- ✓ Continuously improved by thousands of developers worldwide.



TYPES OF KERNEL

Monolithic Kernel

All core services run in kernel space. (Linux uses a refined monolithic kernel)

Microkernel

Minimal services in kernel space. Most services run in user space.

Hybrid Kernel

Combines monolithic and microkernel approaches.



Why it matters for DevOps Engineers

Understanding the Linux kernel helps you troubleshoot performance issues, optimize systems, and work confidently with servers, containers, and cloud environments.



Remember

The kernel is invisible, but everything you do on Linux depends on it.

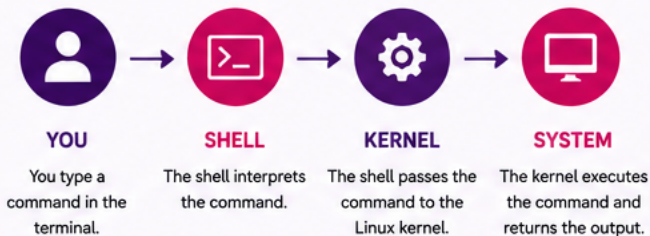


LINUX SHELL

Your Command-Line Powerhouse

The Linux Shell is a command-line interpreter that acts as an interface between you and the Linux kernel. It allows you to execute commands, run scripts, automate tasks and manage your system efficiently.

HOW THE SHELL WORKS



WHAT IS A SHELL?

A shell is a program that:

- ✓ Accepts commands from you
- ✓ Executes programs and utilities
- ✓ Handles input and output
- ✓ Supports scripting and automation
- ✓ Manages the environment



COMMON SHELLS IN LINUX

- ✓ **Bash** (Bourne Again Shell)
Default shell in most distributions
- ✓ **Zsh** (Z Shell)
Feature-rich and customizable
- ✓ **Sh** (Bourne Shell)
Original Unix shell
- ✓ **Fish** (Friendly Interactive Shell)
User-friendly and intelligent



KEY FEATURES

- ✓ Command History
- ✓ Tab Completion
- ✓ I/O Redirection and Pipes
- ✓ Environment Variables
- ✓ Shell Aliases
- ✓ Job Control
- ✓ Scripting Support
- ✓ Extensibility



WHY DEVOPS ENGINEERS NEED THE SHELL

- ✓ Manage servers remotely
- ✓ Automate repetitive tasks
- ✓ Work with DevOps tools (Docker, Kubernetes, Git, Terraform, etc.)
- ✓ Troubleshoot systems quickly
- ✓ Write scripts for deployment and operations



SHELL = PRODUCTIVITY

“The more comfortable you are with the shell, the more powerful and efficient you become as an engineer.”

COMMON SHELL COMMANDS



NAVIGATE

pwd – Print working directory
ls – List files and directories
cd – Change directory



MANAGE FILES

cp – Copy files or directories
mv – Move / rename files
rm – Remove files or directories
mkdir – Create directory
touch – Create empty file



VIEW CONTENT

cat – Display file content
less – View file page by page
head – First 10 lines
tail – Last 10 lines



SYSTEM INFO

uname -a – System information
top – Monitor processes
df -h – Disk usage
free -h – Memory usage



PERMISSIONS

chmod – Change permissions
chown – Change owner
chgrp – Change group
ls -l – Long listing



PIPES & REDIRECTION

command > file – Redirect output
command >> file – Append output
command 2> error.log – Redirect errors
command | another – Pipe output



PRO TIP

Everything in Linux is a file, and the shell is the key to unlocking the full power of the system.



KEEP PRACTICING

Use the shell daily. The more you practice, the more natural and powerful it becomes.



Your journey to DevOps mastery starts with the shell.

Learn it. Love it. Live it.



UNDERSTANDING LINUX SERVERS

The backbone of modern IT infrastructure



What is a Linux Server?

A Linux server is a computer system running the Linux operating system, designed to provide services, manage resources and handle requests from clients over a network.

- ✓ Stable
- ✓ Secure
- ✓ Scalable
- ✓ High Performance
- ✓ Cost-Effective
- ✓ Open Source



KEY COMPONENTS OF A LINUX SERVER



Hardware

Physical machine or virtual instance with CPU, RAM, storage and network.



Linux Kernel

Core of the OS. Manages hardware, processes, memory, files and more.



System Services

Background services that provide functionality (web, database, mail, DNS).



File System

Organizes and stores data in a structured hierarchy.



Users & Permissions

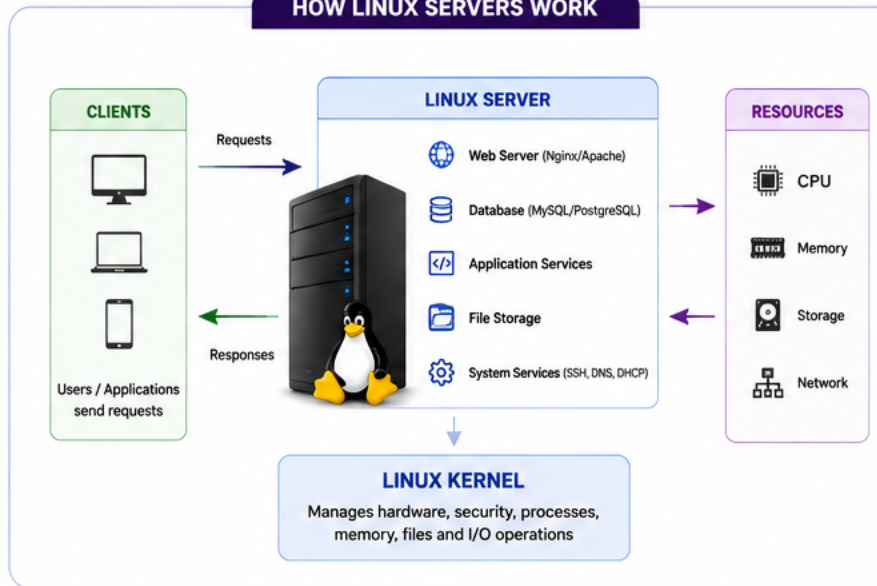
Controls who can access what and what actions they can perform.



Network

Enables communication with clients, other servers and the internet.

HOW LINUX SERVERS WORK



WHY CHOOSE LINUX SERVERS?

- ✓ **Reliable & Stable**
Designed for 24/7 uptime and heavy workloads.
- ✓ **Secure**
Strong permission model, regular updates, and active security community.
- ✓ **Flexible**
Supports a wide range of software and use cases.
- ✓ **Scalable**
Easily scales from small projects to enterprise infrastructure.
- ✓ **Cost-Effective**
Open source with no licensing fees.
- ✓ **Powerful Community**
Huge global community and documentation support.



BENEFITS OF LINUX SERVERS



Better Security



High Performance



Lower Total Cost



Highly Customizable



Ideal for Cloud & DevOps



COMMON USE CASES

- Web Hosting
- Databases
- Email Servers

- Cloud & Virtualization
- DevOps / CI-CD
- File & Print Servers

- DNS & DHCP Servers
- Streaming Servers
- Data Processing



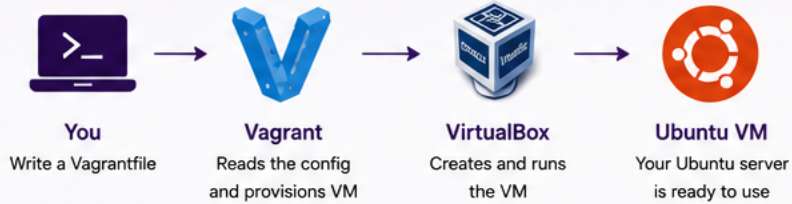
TIPS FOR MANAGING LINUX SERVERS

- ✓ Keep your system updated
- ✓ Use strong passwords and SSH keys
- ✓ Follow the principle of least privilege
- ✓ Monitor system resources and logs
- ✓ Automate tasks with scripts and tools

Install Ubuntu using Vagrant and VirtualBox

Vagrant helps you automate the creation and management of virtual machines. VirtualBox is the virtualization platform that runs the VM.

HOW IT WORKS



STEP-BY-STEP GUIDE

1 Install Prerequisites



- Install VirtualBox from [virtualbox.org](https://www.virtualbox.org)
- Install Vagrant from [vagrantup.com](https://www.vagrantup.com)



2 Create a Project Directory



Open your terminal and run:

```
mkdir ubuntu-vagrant
cd ubuntu-vagrant
```

3 Initialize Vagrant



Run the command:

```
vagrant init
ubuntu/jammy64
```

This creates a **Vagrantfile** in your directory.

4 (Optional) Edit Vagrantfile



Open **Vagrantfile** and ensure the box is set:

```
config.vm.box = "ubuntu/jammy64"
config.hostname = "ubuntu-vm"
config.vm.network "private_network",
  ip: "192.168.56.10"
config.vm.provider "virtualbox" do |vb|
  vb.memory = "2048"
  vb.cpus = 2
end
```

This sets memory, CPU and a private IP.

5 Start the VM



Run the command:

```
vagrant up
```

Vagrant will:

- ✓ Download the Ubuntu box (if not already present)
- ✓ Create the VirtualBox VM
- ✓ Provision and start the VM

6 SSH into Ubuntu VM



Run the command:

```
vagrant ssh
```

You are now logged in to your Ubuntu VM!

VERIFY THE INSTALLATION

Once inside the VM, check:

```
$ lsb_release -a          $ uname -r
No LSB modules are available.  5.15.0-105-generic
Distributor ID: Ubuntu
Description: Ubuntu 22.04.4 LTS
Release: 22.04
Codename: jammy           $ ip a | grep inet
                             inet 192.168.56.10/24 brd
                             192.168.56.255 scope global
                             dynamic noprefixroute enp0s8
```

USEFUL VAGRANT COMMANDS

vagrant up	Start the VM
vagrant halt	Stop the VM
vagrant reload	Restart the VM
vagrant ssh	SSH into the VM
vagrant status	Check VM status
vagrant destroy	Delete the VM

✓ YOUR UBUNTU VM IS READY!

- ✓ You now have a clean Ubuntu server environment.
- ✓ Use it for learning, development, testing and DevOps labs.
- ✓ Recreate the same environment anytime with a single command:
vagrant up



CONNECTING TO LINUX SERVERS

SSH (Secure Shell) allows you to securely connect to and manage Linux servers over an untrusted network.

HOW SSH CONNECTION WORKS



WHY SSH?

- ✓ Secure authentication
- ✓ Encrypted communication
- ✓ Remote command execution
- ✓ File transfer (SCP/SFTP)
- ✓ Essential for DevOps engineers

STEPS TO CONNECT

1 GET SERVER INFORMATION



- Obtain the server's:
- IP address or Domain
 - Username
 - SSH Port (default: 22)

2 OPEN TERMINAL



Open your terminal (Linux, macOS, or Windows with Git Bash / PowerShell).

3 CONNECT USING PASSWORD



```
ssh username@server_ip
```

```
ssh ubuntu@192.168.56.10
```

You will be prompted for the password.

4 CONNECT USING SSH KEY (RECOMMENDED)



```
ssh -i /path/to/private_key username@server_ip
```

```
ssh -i ~/.ssh/mykey.pem ubuntu@192.168.56.10
```

More secure and convenient.

5 YOU ARE NOW CONNECTED!



If successful, you'll see the welcome message and command prompt.

```
ubuntu@server:~$
```

6 DISCONNECT



Type the command to exit the server session.

```
exit
```

or press Ctrl + D

SSH KEY AUTHENTICATION (BEST PRACTICE)

- Generate SSH Key Pair


```
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
```
- Copy Public Key to Server


```
ssh-copy-id -i ~/.ssh/id_rsa.pub username@server_ip
```
- Connect Using SSH Key


```
ssh -i ~/.ssh/id_rsa username@server_ip
```
- Disable Password Login (Optional but Recommended)
 Edit /etc/ssh/sshd_config and set PasswordAuthentication no

KEY BENEFITS

- No password typing
- More secure
- Automated access
- Ideal for DevOps & CI/CD

USEFUL SSH COMMANDS

<code>ssh user@host</code>	Connect to a server
<code>ssh -i key.pem user@host</code>	Connect with a private key
<code>ssh user@host -p 2222</code>	Connect using custom port
<code>scp file.txt user@host:/path/</code>	Copy file to server
<code>scp user@host:/path/file.txt .</code>	Copy file from server
<code>sftp user@host</code>	Secure file transfer
<code>ssh -t user@host "command"</code>	Execute remote command

BEST PRACTICES

- ✓ Always use SSH keys instead of passwords.
- ✓ Use strong passphrases for your keys.
- ✓ Change the default SSH port if possible.
- ✓ Keep your system and SSH server updated.
- ✓ Do not share your private key.
- ✓ Use a non-root user with sudo privileges.
- ✓ Enable fail2ban or similar protection.

Security + Best Practices = Strong Servers



UNDERSTANDING THE LINUX DIRECTORY STRUCTURE

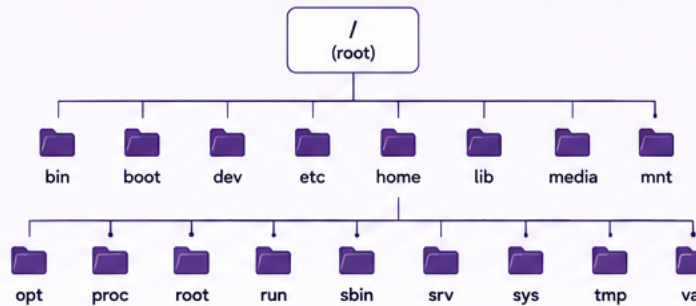
Linux organizes everything in a single directory tree starting from the root (/). Understanding this structure is essential for navigating and managing Linux systems.



Everything in Linux is a file

Devices, directories, configuration, and data are all treated as files.

THE LINUX DIRECTORY TREE



The root (/) is the base of the filesystem hierarchy. All other directories branch from it.



KEY POINTS

- ✓ Root directory (/) is the starting point of the entire filesystem.
- ✓ Linux follows the Filesystem Hierarchy Standard (FHS).
- ✓ Directories have specific purposes and contain different types of files.
- ✓ Understanding the structure helps you work efficiently and avoid accidental changes.

COMMON DIRECTORIES AND THEIR PURPOSES

Directory	Purpose	Example Contents	Directory	Purpose	Example Contents
/	Root of the entire filesystem	Contains all other directories	/media	Mount point for removable media	USB drives, CDs, external disks
/bin	Essential user command binaries	ls, cp, mv, cat, echo	/mnt	Temporary mount point	Manually mounted filesystems
/boot	Boot loader files and kernel	vmlinuz, initrd, grub	/opt	Optional third-party software	/opt/google, /opt/docker
/dev	Device files	sda, tty, null, random	/proc	Virtual filesystem with process info	CPU info, running processes
/etc	System configuration files	passwd, fstab, ssh, network config	/root	Home directory for the root user	Root user's files
/home	User home directories	/home/ubuntu, /home/devops	/usr	User programs and data	bin,/sbin,lib,share,local
/lib	Essential shared libraries	libc.so, kernel modules	/var	Variable data files	logs, spool, cache, mail
			/tmp	Temporary files	Temporary files (cleared on reboot)
			/sys	System and kernel information	Kernel, devices, drivers info
			/srv	Data for services provided by system	Web, FTP, DNS data
			/run	Runtime data since last boot	PID files, sockets



QUICK TIP

Most Linux commands and configuration files are stored under /etc, binaries under /bin or /usr/bin, and user data under /home.

NAVIGATION TIPS



List directory contents

```
ls
ls -l (long format)
ls -la (including hidden files)
```



Change directory

```
cd /path/to/directory
cd .. (go up one level)
cd ~ (go to home directory)
```



Print working directory

```
pwd
```



Explore a directory

```
ls /path/to/directory
```



Practice navigating using the terminal to become comfortable with the Linux filesystem.



WORKING WITH FILES AND DIRECTORIES

Linux treats everything as a file, and directories are special files that contain other files. These commands help you manage them efficiently.

FILES vs DIRECTORIES



FILES

Store data, content, configuration and programs.

Examples: file.txt, script.sh, report.pdf



DIRECTORIES

Organize files and other directories in a hierarchy.

Examples: docs/, projects/, images/

COMMON CONCEPTS



Home Directory (~)

Your personal workspace.



Dot Files (.file)

Hidden files and directories whose names start with a dot.



Path

Location of a file or directory. Can be absolute or relative.



/ (Root)

The top of the directory hierarchy. Everything starts here.



ESSENTIAL COMMANDS



1. LIST CONTENTS

List files and directories in the current directory.

```
ls
ls -l      # long format
ls -a      # show hidden
ls -lh     # human readable
```



2. CHANGE DIRECTORY

Move to a different directory.

```
cd /path/to/directory
cd ..      # go up one level
cd ~       # go to home
cd -       # go to previous dir
```



3. MAKE DIRECTORY

Create a new directory.

```
mkdir dirname
mkdir -p parent/child/grandchild
# create parent directories too
```



4. REMOVE DIRECTORY

Delete an empty directory. Use -r to remove with contents.

```
rmdir dirname
rm -r dirname      # remove
# directory with contents
```



5. CREATE FILE

Create a new empty file or update timestamp.

```
touch filename
touch -a filename # access time
touch -m filename # modify time
```



6. COPY

Copy files or directories.

```
cp file.txt /dest/
cp -r dir1/ dir2/ # copy
# directory recursively
```



7. MOVE / RENAME

Move or rename files and directories.

```
mv oldname newname # rename
mv file.txt /new/path/ # move
```



8. REMOVE FILE

Delete a file.

```
rm filename
rm -f filename # force delete
rm -i filename # interactive
```



9. VIEW FILE CONTENT

Display file contents in the terminal.

```
cat filename # all content
less filename # page by page
head filename # first 10 lines
tail filename # last 10 lines
```



10. FIND FILES

Search for files and directories.

```
find /path -name "filename"
find . -type d # find dirs
find . -type f -name "*.txt"
```

WILDCARDS

Use wildcards to match patterns:

- * matches any number of characters e.g. *.txt
- ? matches any single character e.g. file?.txt
- [abc] matches any one of the characters e.g. file[1-3].txt
- ~ your home directory e.g. cd ~/docs

EXAMPLE WORKFLOW

1. Create a directory
2. Navigate into it
3. Create a file
4. Copy file
5. View file
6. Move file
7. Remove file
8. Go back

```
$ mkdir projects
$ cd projects
$ touch notes.txt
$ cp notes.txt notes_backup.txt
$ cat notes.txt
$ mv notes.txt ideas.txt
$ rm ideas.txt
$ cd ..
```

GOOD PRACTICES

- ✓ Use clear and meaningful names.
- ✓ Organize files in logical directories.
- ✓ Use -i with rm to avoid accidental deletion.
- ✓ Use tab completion to save time.
- ✓ Always check paths before deleting or moving files.



TIP: Everything in Linux is case-sensitive. File.txt, file.txt and FILE.txt are different!



LINUX TEXT EDITORS

Text editors in Linux are powerful tools used to create, edit and manage text files, configuration files and code.



OVERVIEW

Linux offers a variety of text editors. The most commonly used are Vim, Nano and Emacs.

- ✓ Edit configuration files
- ✓ Write scripts and code
- ✓ Manage system files
- ✓ Available on almost all Linux systems



QUICK COMPARISON

Editor	Best For	Learning Curve	Features
Vim	Power users, admins, devs	★★★☆☆ (Moderate)	Fast, powerful, keyboard-driven
Nano	Beginners, quick edits	★☆☆☆☆ (Easy)	Simple, user-friendly
Emacs	Developers, advanced users	★★★★☆ (High)	Extensible, all-in-one environment



VIM

Highly efficient, keyboard-driven editor

KEY FEATURES

- Modes: Normal, Insert, Visual, Command
- Very powerful for programmers and sysadmins
- Works great on remote servers
- Extremely customizable

BASIC COMMANDS

```
vim filename      # open file
i                 # insert mode
Esc               # back to normal mode
:w                # save
:q                # quit
:wq               # save and quit
:q!               # quit without saving
/word             # search forward
?word             # search backward
```



Vim has a steep learning curve but is extremely powerful once mastered.



NANO

Simple and easy-to-use editor

KEY FEATURES

- Easy to learn and use
- Shows helpful shortcuts at the bottom
- Great for quick edits and beginners
- Comes installed on most Linux systems

BASIC COMMANDS

```
nano filename     # open a file
Ctrl + O          # write out (save)
Enter             # confirm filename
Ctrl + X          # exit
Ctrl + K          # cut line
Ctrl + U          # paste line
Ctrl + W          # search
Ctrl + G          # help
```



Nano is perfect for beginners and quick configuration changes.



EMACS

Extensible, powerful, and beyond just an editor

KEY FEATURES

- Extremely powerful and extensible
- Supports plugins and scripting (Emacs Lisp)
- Built-in tools: email, terminal, file manager, more
- Ideal for developers and power users

BASIC COMMANDS

```
emacs filename    # open a file
Ctrl + X Ctrl + S # save
Ctrl + X Ctrl + C # exit
Ctrl + X Ctrl + F # open file
Ctrl + G          # cancel command
Ctrl + S          # search forward
Ctrl + R          # search backward
F1                # help
```



Emacs can be a full IDE and productivity suite inside your terminal.



WHEN TO USE WHICH?



Use Vim when you want speed, precision and are comfortable with keyboard shortcuts.



Use Nano for quick edits, simple tasks and when you are new to Linux.



Use Emacs when you need a powerful, extensible environment for development and more.



GENERAL TIPS

- ✓ Always save your work before exiting.
- ✓ Use `man <editor>` to read the manual and learn more.
- ✓ Practice regularly to improve speed and confidence.
- ✓ Backup important files before editing.
- ✓ Learn one editor deeply before moving to another.

